FUNDAMENTALS IN AIML

PROJECT REPORT

AlgoPilot

Name: Shambhavi Shanker

Date: 30.03.26

Registration Number: 25BHI10120

Subject: Fundamentals in AI/ML

INDEX

INTRODUCTION 2
PROBLEM STATEMENT 3
OBJECTIVES 4
METHODOLOGY 5
DESIGN DIAGRAM 7
IMPLEMENTATION DETAILS 8
RESULT/SCREENSHOTS 9
TESTING APPROACH 10
CHALLENGES FACED 11
LEARNINGS 12
FUTURE ENHANCEMENTS 13
CONCLUSION 14
REFERENCES 14

INTRODUCTION

AlgoPilot is a Python based system that helps choose the most suitable pathfinding algorithm for a given maze. It works with three common algorithms Breadth First Search (BFS), Depth First Search (DFS), and A*and analyzes which one performs best based on the structure of the maze.

In problem solving, multiple algorithms can be used to reach a solution, but their efficiency depends on the situation. For example Breadth-First Search is really good at finding the path but it has to look at a lot of nodes. Depth-First Search is fast. It does not always find the best path.. A* is really good for big problems or problems where you have a specific goal in mind. This makes it hard to decide which algorithm to use for a problem.

The AlgoPilot project is trying to solve this problem by using machine learning. It looks at things like how dense the maze's how open it is if there are a lot of dead ends and how far apart things are. Then it uses this information to predict which algorithm will work the best for that maze.

The system also explains why a particular algorithm is chosen and visualizes the final path. This makes it not only useful for solving problems but also helpful for students to understand how different algorithms behave in different situations.

PROBLEM STATEMENT

Pathfinding algorithms like BFS, DFS, and A* are something we learn in class, but it’s not always clear which one to actually use in a given situation. Each of them behaves differently depending on how the maze or environment looks.

In most cases, we study these algorithms separately as part of the syllabus, but we don’t really get to see how they compare in real situations. This makes it harder to understand when and why one algorithm is better than another.

So there is a need, for a system that can take a maze analyze it and suggest which algorithm would work best. At the time it should also explain its decision so that it actually helps in understanding the algorithms better not just giving an answer.

OBJECTIVES

The main objectives of this project are:

- To compare how BFS, DFS, and A* perform on different mazes
- To understand how the structure of a maze affects algorithm performance
- To use machine learning to predict the best algorithm for a given maze
- To explain why a particular algorithm is chosen
- To show the path taken by the selected algorithm
- To help students understand these algorithms better.

METHODOLOGY

The project was developed using the following steps:

1. **Maze Generation**
   Random mazes were created with different structures by varying the number of walls and open spaces. This ensured a variety of test cases.
2. **Running Algorithms**
   We applied three algorithms. Breadth-First Search (BFS) Depth-First Search (DFS) and A*. To each maze to find a path from the start to the end. We used each algorithm on every maze.
3. **Performance Comparison**
   The algorithms were compared based on the number of steps taken to reach the goal.
4. **Feature Extraction**
   Important features of the maze were calculated, such as density, openness, number of dead ends, and distance between start and end.
5. **Dataset Creation**
   A dataset was created by storing the features of each maze along with the algorithm that performed best.
6. **Model Training**
   A machine learning model (Random Forest) was trained using this dataset to learn patterns between maze features and the best algorithm.
7. **Prediction**
   The trained model was used to predict the best algorithm for new mazes.
8. **Explanation and Visualization**
   The system explains why it made a prediction and shows the path the chosen algorithm took.

RESULT/SCREENSHOTS

```
(base) shambhavi@Shambhavis-MacBook-Air AlgoPilot % python main.
py
1. Generate random maze
2. Enter your own maze
3. Exit
Enter choice: 1

Maze:
0 0 1 0 1 1 0 1 0 0
0 0 0 0 1 0 1 0 1 1
1 1 1 0 1 1 0 0 0 0
0 0 1 0 0 0 0 0 0 0
0 0 1 0 1 0 1 0 0 0
1 1 1 0 0 1 0 1 0 0
0 1 1 0 0 0 1 0 0 0
0 0 0 0 0 0 0 0 0 0
0 1 1 0 0 0 0 0 0 0
1 0 1 0 0 0 0 1 0 0

Start: (0, 0)
End: (9, 9)

Algorithm Comparison
BFS  -> Steps: 60, Time: 0.00010395050048828125
DFS  -> Steps: 111, Time: 0.00015687942504882812
A*   -> Steps: 48, Time: 0.00017189979553222656
 Model Performance
Accuracy: 96.67%

Most common best algorithm in data:
- bfs: 100 cases
- astar: 100 cases
- dfs: 100 cases

Main factors affecting the decision:
- density: 0.26
- openness: 0.3
- dead_ends: 0.05
- distance: 0.13
- branching: 0.18
- corridor: 0.08

Predicted best algorithm: astar

Why this algorithm?
- A* is best when goal direction matters
- It avoids unnecessary exploration

Quick summary:
- Walls (density): 0.3
- Open space: 0.7
- Dead ends: 0.1
- Start to end distance: 18
```

A randomly generated maze along with performance comparison of BFS, DFS, and A*. The model predicts the best algorithm based on extracted features.

```
Path Visualization
* * 1 0 1 1 0 1 0 0
0 * * * 1 0 1 0 1 1
1 1 1 * 1 1 0 0 0 0
0 0 1 * * * * * * *
0 0 1 0 1 0 1 0 0 *
1 1 1 0 0 1 0 1 0 *
0 1 1 0 0 0 1 0 0 *
0 0 0 0 0 0 0 0 0 *
0 1 1 0 0 0 0 0 0 *
1 0 1 0 0 0 0 1 0 *
```

Visualization of the path generated by the selected algorithm, clearly illustrating the sequence of steps taken from the start to the goal. This helps in understanding how the algorithm navigates through the maze, avoids obstacles, and reaches the solution efficiently.

```
Maze:
0 0 0 0 0 1 0 0 0 0
0 0 0 0 0 1 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 1 0
0 0 0 0 1 1 0 0 1 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 1 0 0 0

Start: (0, 0)
End: (3, 0)

Algorithm Comparison
BFS  -> Steps: 7, Time: 1.3113021850585938e-05
DFS  -> Steps: 70, Time: 0.00010228157043457031
A*   -> Steps: 4, Time: 3.123283386230469e-05
 Model Performance
Accuracy: 96.67%

Most common best algorithm in data:
- bfs: 100 cases
- astar: 100 cases
- dfs: 100 cases

Main factors affecting the decision:
- density: 0.32
- openness: 0.28
- dead_ends: 0.05
- distance: 0.12
- branching: 0.11
- corridor: 0.12

Predicted best algorithm: bfs

Why this algorithm?
- BFS guarantees shortest path
- Good for simple/open mazes

Quick summary:
- Walls (density): 0.07
- Open space: 0.93
- Dead ends: 0.0
- Start to end distance: 3

Path Visualization
* 0 0 0 0 1 0 0 0 0
* 0 0 0 0 1 0 0 0 0
* 0 0 0 0 0 0 0 0 0
* 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 1 0
```

Example showing the model predicting a different algorithm (BFS) for a simpler maze, indicating that the system adapts based on input characteristics.

```
(base) shambhavi@Shambhavis-MacBook-Air AlgoPilot % python main.py
3. Exit
Enter choice: 2

--- Custom Maze Input ---
Number of rows (e.g., 3): 3
Number of columns (e.g., 3): 3

Enter the maze row by row:
Use 0 = open path, 1 = wall
You can enter like: 0 0 1  OR  001

Row 1: 001
Row 2: 010
Row 3: 000

Enter START position
Format: row col  (example: 0 0)
Start: 0 0

Enter END position
Format: row col  (example: 2 2)
End: 2 2

Maze:
0 0 1
0 1 0
0 0 0

Start: (0, 0)
End: (2, 2)

Algorithm Comparison
BFS  -> Steps: 6, Time: 3.0040740966796875e-05
DFS  -> Steps: 9, Time: 2.002716064453125e-05
A*   -> Steps: 6, Time: 3.933906555175781e-05
 Model Performance
Accuracy: 98.33%

Most common best algorithm in data:
- bfs: 100 cases
- astar: 100 cases
- dfs: 100 cases

Main factors affecting the decision:
- density: 0.29
- openness: 0.27
- dead_ends: 0.06
- distance: 0.13
- branching: 0.14
- corridor: 0.11

Predicted best algorithm: astar
```

The system was also tested using user-defined maze inputs, where the user manually provides the maze along with start and end positions. This shows that the system can handle custom scenarios beyond randomly generated cases.

USE OF ML IN ALGOPILOT

The project uses machine learning to figure out which search algorithm is best for a problem. For each maze it looks at things like how big the grid's how many obstacles are in the way and how far the start and goal are from each other. It also collects information about how algorithms like BFS, DFS and A* work, such as how long they take and how many steps they need.

This information is then used to teach a model that can learn which algorithm works best in different situations. When it gets a maze the model can predict which algorithm is the most suitable so it does not have to try all of them every time.

The main goal of the project is to show how machine learning can be used in a way to help make decisions and reduce guessing when solving problems rather than trying to build a complicated machine learning system. The project uses machine learning to help with this. The idea is to use machine learning to make things easier. The project does this by using machine learning to choose the best search algorithm, for each maze.

TESTING APPROACH

The system was checked by making a lot of environments with different sizes and lots of obstacles. Each maze was solved using the BFS method the DFS method and the A* method. We looked at how they did based on how long it took and how many points they looked at. This data was recorded and used to train the machine learning model.

After training, the model was tested on new unseen mazes. For each test the system first figured out which method would work best and then used it. The results that the system predicted were compared to the method that actually worked best to see how accurate it was.

We used a lot of test cases to make sure the system was working correctly. We also looked at special cases, like maze environments with a lot of obstacles and small grids. This way we could confirm that the methods were correct and that the machine learning model was working well. The maze environments and the BFS method and the DFS method and the A* method were all important to make sure the system was working properly.

CHALLENGES FACED

One of the main challenges in this project was ensuring that the generated mazes were valid and had a possible path between the start and goal positions. Sometimes the random generator would make a maze with no way so we had to fix that or make a new one.

Another difficulty was correctly implementing and debugging the search algorithms, especially A*, where maintaining the priority queue and heuristic calculations needed careful attention.

The A* algorithm and other algorithms like BFS and DFS had to be treated when we were collecting data. At first the A* algorithm was doing a lot better so the model was predicting it would be the choice most of the time.

We had to make sure the data was balanced so the model would not be biased. We had to check the results carefully.

It also took a lot of time to get all the parts of the project working together smoothly. These parts included making the mazes running the algorithms collecting the data and making predictions.

We wanted to keep the system simple. We also wanted it to be able to learn in a meaningful way and that was a challenge.

LEARNINGS

This project really helped me understand how different search algorithms work in situations. I found out that no single search algorithm is the best in every case and how well it works depends a lot on the problem. The project also gave me hands on experience with using machine learning to make decisions not to make predictions. I learned a lot from working with features. Seeing how they affect how well the model works.

The search algorithms and machine learning were important to this project. I was able to use what I learned about search algorithms and machine learning to make the project better.

Working with search algorithms and machine learning and handling data and training models all together in one system really helped me get better at solving problems. It also showed me how important it is to keep the code clean and organized like with code structure and modular design because when the project is organized it is easier to work with the different parts. The search algorithms and machine learning were key, to making this project work.

CONCLUSION

This project shows how machine learning can help us make choices when picking algorithms. We can use search algorithms and a simple learning model together to guess which strategy will work best for a problem. This way we do not have to try lots of things to see what works. The project shows that using data to make decisions is an idea.

The system is not too complicated. It does a good job of showing how different artificial intelligence ideas can be used together in one program.

The machine learning project can be made complicated, with harder problems and better models but for now it does what it is supposed to do: it builds a system that can adapt and solve problems in a smart way.

REFERENCES

- Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, Pearson.
- scikit-learn Documentation.
- Python Documentation.
- GeeksforGeeks – BFS, DFS, and A* Algorithms.